

8. 호이스팅

호이스팅이란, 코드를 실행하기 전에 내부에서 변수와 함수의 위치를 맨 위로 옮겨 선언하는 자바스크립트의 문법적인 기능으로 함수 선언식은 호이스팅 대상입니다.

- 모든 변수는 함수 본문 어느 부분에서 선언 되더라도 내부적으로 함수의 맨 윗부분으로 끌어올려진다.
- 함수도 변수에 할당 되는 객체이기 때문에 동일하게 적용이 된다.

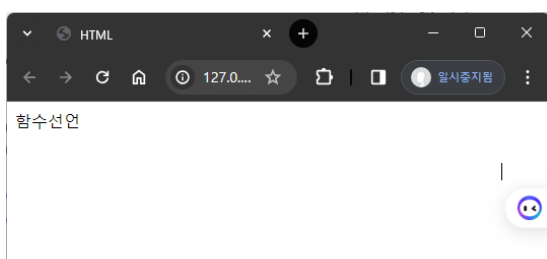
```
<body>
  <Label id="labelDeclare" ></Label>
  <Label id="labelExpress" ></Label>
</body>
<script>
  fnOnClickDeclare();
  fnOnClickExpress();

  function fnOnClickDeclare() {
    document.querySelector("#labelDeclare").innerHTML = "함수선언";
  }

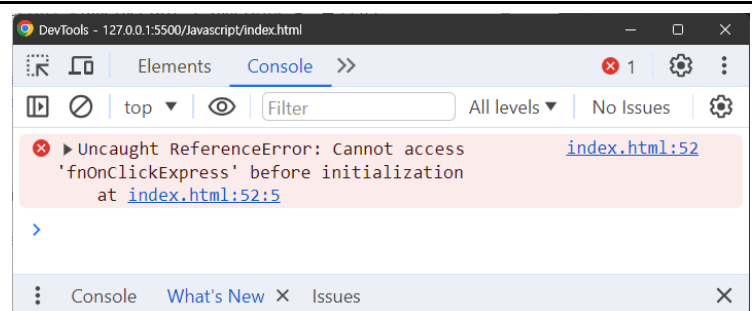
  const fnOnClickExpress = () => {
    document.querySelector("#labelExpress").innerHTML = "함수표현";
  }
</script>
```

웹 브라우저를 통해서 보면 "함수선언"만 화면에 표시 되고 "함수표현"은 표시가 되지 않았습니다.

실행화면



개발자 도구에서 오류



처리 순서는 되므로 에러에 의해서 "함수표현"이 화면에 표시 되지 않습니다.

1. 페이지가 화면에 로딩 되면서 자바스크립트 변수 및 함수가 먼저 호이스팅 됩니다.
 - fnOnClickDeclare 함수 선언
 - fnOnClickExpress 변수 선언 (const fnOnClickExpress)
2. 함수 실행
 - 7번 함수 실행 : fnOnClickDeclare()
 - "함수선언" 화면에 표시
 - 8번 함수 실행 : fnOnClickExpress ()
 - 변수만 선언이 되어 있으므로 에러 발생

- Uncaught ReferenceError: Cannot access 'fnOnClickExpress' before initialization

1. 생명 주기

- 전역 변수의 생명 주기는 애플리케이션의 생명 주기와 같습니다.
- 지역 변수의 생명 주기는 함수의 생명 주기와 일치 합니다.

호이스팅은 스코프를 단위로 일어납니다.

```
> var x = 'global';

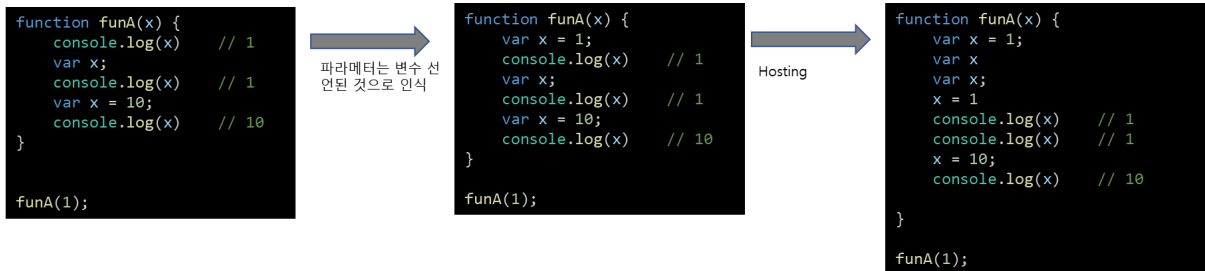
function foo() {
  console.log("내부 x :: " + x);
  var x = 'local';
}

foo();
console.log("외부 x :: " + x);

내부 x :: undefined
외부 x :: global
```

2. 변수 호이스팅

- 변수 선언이 되어 있지 않을 때 : 오류 발생
- 변수가 사용되고 나서 선언이 되었을 때 : 호이스팅
- for 문 초기화 식에서 변수 선언시 호이스팅
 - 변수 선언이 되어 있지 않을 때 : 오류 발생
 - 변수가 사용되고 나서 선언이 되었을 때 : 호이스팅
- if 문 내에서 변수 선언시 호이스팅
 - if 문 외부에 이름이 동일한 변수가 선언되어 있을 때 : 호이스팅
 - 변수가 사용되고 나서 선언이 되었을 때 : 호이스팅



```
console.log("변수 :: a = ", a); // 변수 :: a = undefined
var a = 1; ← Hoisting
let b = 1;
const c = 1;
console.log("a :: "+ a , " b :: "+ a, ", c :: "+ c);
// a :: 1 b :: 1 , c :: 1
```

2-1. var 함수 스코프

- 변수 중복 허용 : 5 라인 x, 7라인 y는 무시 됩니다.\

```
{% code lineNumbers="true" %}
```

```
var x = 1;
var y = 1;

// var 키워드 무시
var x = 100;
// 변수 선언 무시
var y;

console.log(x); // 100
console.log(y); // 1
```

```
{% endcode %}
```

- 함수 레벨 스코프

```
var x = 1;
if(true) {
    var x = 100;
}
```

```
console.log(x); // 100
```

- 변수 호이스팅\

```
{% code lineNumbers="true" %}
```

```
// 1. 선언 단계 : 이미 fnFoo 선언되어 있음
// 2. 초기화 단계 : fnFoo undefined로 초기화
console.log(fnFoo); // undefined

// 변수에 값을 할당 (3. 할당 단계)
fnFoo = 123;

console.log(fnFoo); // 123

// 런타임 이전에 암묵적 실행
var fnFoo;
```

```
{% endcode %}
```

2-2. let, const 블록 스코프

- 변수 중복 선언 금지\

```
let bar = 123;
let bar = 234; // Uncaught SyntaxError: Identifier 'bar' has already
                been declared
```

- 블록 레벨 스코프\

```
let foo = 1; // 전역 변수
{
  let foo = 2; // 지역 변수
  let bar = 3; // 지역 변수
}

console.log(foo); // 1
console.log(bar); // ReferenceError: bar is not defined
```

\

- 변수 호이스팅 : 선언한 변수는 스코프의 시작 시점부터 초기화 단계 시작 지점(변수 선언문)까지 변수를 참조할 수 없습니다.\

```
let foo = 1; // 전역

{
  console.log(foo); //ReferenceError
  let foo = 2; // 지역
}
```

3. 함수 호이스팅

함수 선언문이 코드의 선두로 끌어올려진 것처럼 동작하는 자바스크립트 고유의 특징을 함수 호이스팅이라 합니다.

함수 선언문으로 정의한 함수와 함수 표현식으로 정의한 함수의 생성 시점이 다르다.

- 함수 선언문으로 정의한 함수를 함수 선언문 이전에 호출하면 함수 호이스팅에 의해 호출이 가능합니다.
 - var 키워드로 선언된 변수는 undefined로 초기화되고, 함수 선언문을 통해 암묵적으로 생성된 식별자는 함수 객체로 초기화됩니다.
- 함수 표현식으로 함수를 정의하면 함수 호이스팅이 발생하는 것이 아니라 변수 호이스팅이 발생합니다.

```
function funcA() {
  console.log(a); // [Function: a]
  var a = 1;
  console.log(a); // 1
  function a() {}
}

funcA();
```

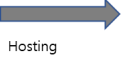


```
function funcA() {
  var a;
  function a() {}

  console.log(a); // [Function: a]
  a = 1;
  console.log(a); // 1
}

funcA();
```

```
function funcA() {
  console.log(a); // undefined
  var a = 1;
  console.log(a); // 1
  var a = function() {};
}
```



```
function funcA() {
  var a ;
  var a ;

  console.log(a); // undefined
  a = 1;
  console.log(a); // 1
  a = function() {}
}

funcA();
```

```
// 함수 참조
console.dir(add); // f add(x,y)
console.dir(sub); // undefined

// 함수 호출
console.log(add(2,5)); // 7
console.log(sub(2,5)); // TypeError

// 함수 선언문
function add(x,y) {
  return x+y;
}

// 함수 표현식
var sub = function(x,y) {
  return x-y;
}
```

- 함수 선언문으로 정의한 함수는 함수 선언문 이전에 호출할 수 있습니다.
- 함수 표현식으로 정의한 함수는 함수 표현식 이전에 호출할 수 없습니다.

```
function fFunc01() {
  console.log("gloabl fFunc01() ... ");
}

function fHoisting() {
  console.log(typeof fFunc01); // function :: function fFunc01 <- Hoisting
  console.log(typeof fFunc02); // undefined <- Hoisting
  console.log(typeof fFunc03); // undefined <- Hoisting
  console.log(typeof fFunc04); // function :: function fFunc04 <- Hoisting
  console.log(typeof fFunc05); // function :: function fFunc05 <- Hoisting
  fFunc01(); // local fFunc01() ...
  // fFunc02(); // TypeError: fFunc02 is not a function
  // fFunc03(); // TypeError: fFunc03 is not a function
  fFunc04(); // local fFunc04() ...
  fFunc05(); // gloabl fFunc05() ...
}
```

```
function fFunc01() { // 재정의 -> 유효범위 체인
    console.log("local fFunc01() ... ");
}
var fFunc02 = function() { // 재정의 -> 유효범위 체인
    console.log("local fFunc02() ... ");
}
var fFunc03 = function () {
    console.log("local fFunc03() ... ");
}
function fFunc04() {
    console.log("local fFunc04() ... ");
}
console.log(typeof fFunc02, " :: ", fFunc02); // function ::
function() {
    fFunc02(); // local fFunc02() ...
}

fHoisting();
function fFunc02() {
    console.log("gloabl fFunc02() ... ");
}

function fFunc05() {
    console.log("gloabl fFunc05() ... ");
}
```